

Arabic Calligraphy Style Recognition

^{1st} Wissam Tedros
MSFEA ECE Dept.
American University of Beirut
Beirut, Lebanon

^{2nd} Ghina Daoud
MSFEA ECE Dept.
American University of Beirut
Beirut, Lebanon

^{3rd} Rami Al-Khatib
MSFEA ECE Dept.
American University of Beirut
Beirut, Lebanon

Abstract—Arabic calligraphy style recognition is a fine-grained visual classification problem where the defining cues, such as stroke curvature, slant, letter proportions, and thickness variation, are subtle, locally distributed, and often ambiguous even to human experts. This project targets automatic recognition of such styles and is directly motivated by the UNESCO recognition of Arabic calligraphy as Intangible Cultural Heritage of Humanity [1]. The current phase has focused on building and validating a robust style classifier using a combined multi-source dataset. The longer-term goal is a two-stage pipeline: (1) a calligraphy style classifier, followed by (2) a style-conditioned OCR module that transcribes the identified text. This report documents progress on stage (1) and outlines the planned direction for stage (2).

I. INTRODUCTION

The value of Arabic calligraphy and the knowledge it has passed down through the generations are reflected in its official recognition by UNESCO [1] as Intangible Cultural Heritage of Humanity. Large collections of Arabic manuscripts and calligraphic works are being digitized concurrently, but there are currently few automatic tools available for comprehending and classifying this material. In order to directly motivate research on style classification with open benchmarks, the creators of the HICMA [2] dataset state that Arabic text recognition in ancient manuscripts and calligraphy paintings is overlooked, that few works use machine learning to detect Arabic styles, and that many existing style datasets are not publicly accessible.

Practical workflows like indexing and classifying digitized pages by style, facilitating search and retrieval in archives, and supplying helpful metadata for document analysis pipelines are made possible by the ability to recognize the calligraphy style. The problem is also made measurable and feasible by using publicly available datasets such as HICMA, which allow models to be trained and assessed consistently under realistic variation.

Since the initial proposal, the team has completed the following: reviewed relevant literature on both calligraphy style recognition and Arabic OCR; collected and combined three datasets (HICMA, DuwatBench, and Ruf’a) spanning seven calligraphic styles; implemented a class-aware preprocessing and augmentation pipeline producing a merged dataset of 23,052 images; designed and trained a ResNet18 local-global fusion classifier; and conducted two comparative training experiments. The style classifier is now stable and validated. The next phase targets OCR integration.

II. LITERATURE REVIEW

Compared to Arabic printed-font recognition, there is still a dearth of work on Arabic calligraphy style/type recognition, primarily due to the handwritten nature, high degree of variation, and fine-grained appearance of calligraphy. A lot of early systems use the traditional document-analysis pipeline, which consists of preprocessing, manually extracted features, and a classifier. For instance, Allaf and Al-Hammuz [3] suggest an offline recognition method that applies a neural network classifier after extracting unique features for every type of calligraphy and optimizing the feature set and image size using a genetic algorithm. Hussein [4] compares single-hidden-layer fast-learning models and suggests a texture-driven framework with four feature types.

More recent research introduces descriptors that are closer to those used by human experts and highlights the fact that generic features are frequently insufficient for fine-grained style separation. In order to distinguish styles, Kaoudja et al. [5] suggest a computational tool that is modeled after expert “indices.” This tool transforms these cues into a descriptor and performs classification using decision fusion. They assess resilience to images taken from various sources and nine calligraphy styles. Simultaneously, deep representation learning has been documented in the literature, particularly for feature learning with limited data. Al-Hmouz [6] suggests a stacked autoencoder system that identifies the font by combining decisions from word or segment crops taken from a calligraphy image, demonstrating high recognition rates on both public and local datasets.

The lack of commonly used public benchmarks, which makes comparison and generalization claims more difficult, is a problem that has persisted throughout the field. Recent dataset releases that specifically target calligraphy have improved this: HICMA [2] offers 5,031 real-world manuscript and calligraphy images in five styles with labels and benchmarking tools in an effort to facilitate reproducible evaluation. In order to address the online environment, Calliar [7] releases 2,500 stroke-trajectory samples that are annotated at the stroke, character, word, and sentence levels. This allows for learning from pen dynamics instead of just static images.

Regarding the planned OCR stage, existing Arabic OCR systems were also reviewed. The core challenge is that Arabic OCR is substantially harder than Latin-script OCR due to the script’s cursive nature, context-dependent letter shapes,

complex ligatures, and diacritical marks [8]. Critically, all reviewed OCR systems — including the LSTM-based Tesseract engine [9], the deep-learning library EasyOCR [10], and the transformer encoder-decoder model TrOCR [11] — were designed and evaluated on printed or handwritten modern Arabic text, not on artistic calligraphy. The closest existing work is HATFormer [12], which fine-tunes TrOCR for historical Arabic manuscripts, but even historical manuscripts differ significantly from artistic calligraphy in terms of stroke deformation and stylistic variation. No OCR system has been specifically benchmarked on multi-style artistic Arabic calligraphy, which confirms that fine-tuning on a paired calligraphic transcription dataset — currently being constructed — is a prerequisite before any meaningful OCR integration can begin.

III. DATASETS

The effectiveness of deep learning models for Arabic calligraphy style classification is fundamentally dependent on dataset quality, stylistic diversity, annotation reliability, and class balance. Unlike general object recognition benchmarks, publicly available Arabic calligraphy datasets remain limited in size and are often fragmented across independent research efforts. Furthermore, many datasets were originally developed for handwriting recognition rather than explicit calligraphy style classification, which introduces additional challenges in dataset selection and preprocessing.

In this project, we evaluate multiple candidate datasets based on the following criteria:

- Explicit availability of calligraphy style labels
- Number of classes and class balance
- Image realism and manuscript authenticity
- Public accessibility and reproducibility

The datasets considered are described below.

A. Publicly Available Datasets

1) *HICMA Dataset*: The Handwriting Identification for Calligraphy and Manuscripts in Arabic (HICMA) dataset is currently the most suitable publicly available dataset for supervised Arabic calligraphy style classification. The dataset is accessible at <https://hicma.net/dataset.html> and is released under a CC BY-NC 4.0 license.

a) *Dataset Content and Annotation*: HICMA contains a total of 5,031 labeled calligraphic sentence images across five artistic styles:

- Naskh
- Thuluth
- Diwani
- Kufic
- Muhaqaq

Each image is annotated with an explicit style label, making it directly suitable for supervised multi-class classification. The images are extracted from authentic manuscripts and digital calligraphy artworks, preserving background texture, page context, and stylistic variability.

b) *Dataset Statistics and Distribution*: The HICMA dataset contains 5,031 annotated calligraphic sentence images distributed across five artistic styles: Naskh, Thuluth, Diwani, Kufic, and Muhaqaq. The dataset exhibits moderate class imbalance, with Naskh representing the largest proportion of samples and Muhaqaq being the least represented class. The distribution of samples across the five styles is illustrated in Figure 1. This imbalance is explicitly considered during model training through stratified data splitting and class-weighted loss functions to ensure fair representation across styles.

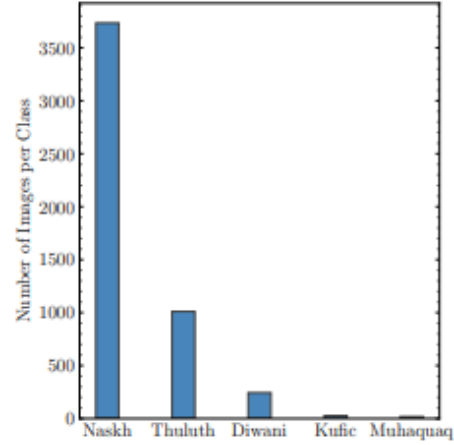


Fig. 1: Distribution of images across the five calligraphy styles in the HICMA dataset.

Representative samples from each style are shown in Figure 2, highlighting the structural diversity, stroke density variation, and stylistic complexity inherent to artistic Arabic calligraphy. The dataset preserves realistic manuscript backgrounds, varying illumination conditions, and heterogeneous text lengths, which increases the difficulty of the classification task and better reflects real-world cultural heritage documents.



Fig. 2: Representative examples from each calligraphic style in the HICMA dataset.

c) *Relevance to the Project*: HICMA directly aligns with the objective of artistic calligraphy style classification due to:

- Explicit style annotations
- Real-world manuscript complexity
- Multiple artistic classes

Therefore, HICMA will serve as the primary dataset for training and evaluation.

2) *DuwaBench Dataset*: The DuwaBench dataset is a benchmark dataset for Arabic handwriting recognition, specifically targeting supplication (Du'a) texts written in diverse calligraphic and handwriting styles. It provides a structured evaluation framework for Arabic script recognition across varying writing conditions and styles.

a) *Dataset Content and Annotation*: DuwaBench contains a collection of labeled Arabic handwriting images spanning multiple writing styles and authors. Each image is annotated with its corresponding textual content and style metadata, making it suitable for both recognition and style classification tasks. The images are sourced from real handwritten manuscripts and digitized religious texts, capturing a wide range of pen types, ink densities, and writing substrates. The styles represented in DuwaBench include:

- Naskh
- Ruq'ah
- Thuluth
- Nastaliq
- Freestyle/Mixed

b) *Dataset Statistics and Distribution*: DuwaBench contains annotated Arabic handwriting images distributed across multiple calligraphic styles. Similar to HICMA, the dataset exhibits class imbalance, with certain styles being more prominently represented than others. The distribution of samples across styles is illustrated in Figure 3. This imbalance is accounted for during training through stratified splitting and class-weighted loss functions, consistent with the approach applied to the HICMA dataset.

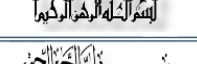
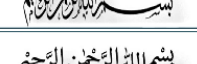
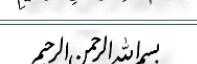
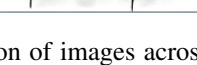
 DuwaBench			
Thuluth		706 samples	567 samples Quranic
Diwani		230 samples	353 samples Devotional/ Hadith
Kufic		83 samples	146 samples Non-religious/ Dedication
Naskh		110 samples	97 samples Names of Allah
Ruqah		76 samples	75 samples Names of Prophet/Companions
Nastaliq		67 samples	34 samples Personal/Place

Fig. 3: Distribution of images across calligraphic styles in the DuwaBench dataset.

Representative samples from each style are shown in Figure 4, illustrating the variability in stroke width, ligature density, and spatial layout across writing styles. The dataset preserves realistic writing conditions including uneven ink distribution, aged paper textures, and inconsistent baseline alignment, increasing classification difficulty and enhancing real-world applicability.



Fig. 4: Representative examples from each calligraphic style in the DuwaBench dataset.

c) *Relevance to the Project*: DuwaBench complements HICMA by introducing:

- A distinct textual domain (religious supplication texts), testing cross-domain generalization of trained models
- Additional calligraphic styles not present in HICMA, broadening style coverage
- Varied writing substrates and ink conditions, increasing robustness evaluation diversity

Therefore, DuwaBench will serve as a secondary dataset for cross-dataset evaluation and generalization testing.

3) *Ruf'a Dataset*: The Ruf'a dataset is an Arabic handwriting dataset centered on the Ruq'ah script style. In the context of this work, only the Ruq'ah class images are used, without any associated text transcriptions or writer metadata. These samples serve solely as additional visual examples of the Ruq'ah style to supplement the training distribution.

a) *Relevance to the Project*: The Ruq'ah class extracted from Ruf'a is incorporated to increase style coverage beyond the classes present in HICMA and DuwaBench. Since Ruq'ah represents one of the most prevalent everyday Arabic handwriting styles yet is absent from the primary datasets, this subset provides the model with exposure to informal, cursive

Arabic script, broadening its generalization capability across both artistic and everyday writing styles.

B. Dataset Pre-processing and Augmentation

1) *Data Preparation and Label Resolution*: All images are loaded as single-channel grayscale arrays prior to processing. The pipeline accepts one or more root dataset directories and recursively discovers every subdirectory containing image files (.jpg, .jpeg, .png). Class labels are resolved through a two-tier lookup: if a `labels.csv` file is present in the image directory or its parent, the `class` field of the corresponding row is used; otherwise, the containing folder name is taken as the class label. Images present in a folder but absent from the associated CSV are silently discarded to avoid orphaned, unannotated samples. Entries carrying comma-separated (mixed-style) class labels are subsequently removed from the final dataset, retaining only unambiguous single-style annotations.

2) *Class-Aware Augmentation Strategy*: To mitigate the class imbalance inherent across the combined datasets, a target-based augmentation policy is applied differentially to minority and majority classes.

a) *Minority Class Over-sampling*: A class is considered a minority class if its sample count falls below $\tau = 0.60 \times N_{\max}$, where $N_{\max}$ is the cardinality of the largest class. For each such class, the number of additional synthetic copies required to reach τ is computed and distributed evenly across all images in that class:

$$k_i = \min\left(\left\lceil \frac{\tau - N_c}{N_c} \right\rceil, k_{\max}\right), \quad k_{\max} = 20 \quad (1)$$

where k_i is the number of augmented copies generated per original image and N_c is the current class count. A minimum of one augmentation per image is enforced whenever a class is selected for over-sampling.

b) *Majority Class Light Augmentation*: Images belonging to classes already at or above τ are subject to a light stochastic augmentation: each image is independently selected for augmentation with probability $p = 0.20$, and if selected, receives $k \sim \mathcal{N}(1.5, 0.5)$ additional copies (rounded and clipped to $[1, 20]$). This introduces mild diversity into the majority classes without materially altering their distribution.

3) *Augmentation Transforms*: All augmented copies are produced using the Albumentations library [13]. The augmentation pipeline applied to each selected image is defined as follows, with each transform applied independently at inference time to produce a single augmented copy:

- **Random Rotation**: rotation uniformly sampled from $[-12, +12]$ with constant-value border fill, applied with probability $p = 0.6$.
- **Elastic Deformation**: elastic transform with displacement field parameters $\alpha = 30$ and $\sigma = 5$, applied with probability $p = 0.3$, simulating natural stroke curvature variation.
- **Stochastic Noise/Blur** (one of the following, $p = 0.5$):
 - Gaussian blur with kernel size ≤ 3
 - Gaussian noise with intensity sampled from $[5, 25]$

– Unsharp-mask sharpening

- **Brightness and Contrast Jitter**: brightness and contrast independently perturbed within ± 0.2 , applied with probability $p = 0.5$.

Following all stochastic transforms, the image is resized back to its original spatial dimensions to ensure consistent input shape across the dataset.

4) *Output Format*: The pipeline produces a flat output directory containing all original and augmented images under a unified naming scheme: original images are saved as `CLASS_0XXXXX.png` and augmented copies as `CLASS_9XXXXX.png`, where the leading digit encodes provenance. A consolidated `labels.csv` is generated alongside, mapping each filename to its class label, providing the complete annotated dataset ready for model training.

5) *Final dataset distribution*: The combined and augmented dataset (fig5) comprises a total of 23,052 labeled images spanning seven Arabic calligraphic styles: Naskh, Ruq'ah, Diwani, Thuluth, Kufic, Muhaqqaq, and Nasta'liq. Of these, 9,540 images are original samples and 13,512 are synthetically augmented copies, representing a 58.6% augmentation ratio. The augmentation pipeline successfully reduced the initial class imbalance, with per-class counts ranging from 1,344 (Nasta'liq) to 5,005 (Naskh). The dataset achieves a Shannon diversity index of 0.969 and a Gini-Simpson index of 0.842, both indicating a well-distributed multi-class collection suitable for training a robust classifier. Classes that were heavily under-represented in the source datasets — particularly Kufic, Muhaqqaq, and Nasta'liq — received the highest augmentation ratios, while majority classes such as Naskh and Ruq'ah underwent only light stochastic augmentation to preserve their natural distribution characteristics.



Fig. 5: Merged Dataset analytics

IV. PROPOSED SYSTEM

(modify later) We designed a two-component system composed of a calligraphy style classification module and an OCR module. Two high-level integration directions were initially considered.

Both directions are motivated by the same observation: Arabic calligraphy styles differ substantially in stroke morphology, spacing patterns, slant, and structural layout, and these variations can influence recognition behavior.

1) *Style Classification Module*: We treated style classification as a fine-grained visual discrimination problem. Given that style differences in Arabic calligraphy are often subtle and local rather than dominated by large global shape differences, a shallow CNN was not selected as the main model. Similarly, handcrafted descriptors such as LPQ, although historically useful for local texture cues, were not adopted as the primary representation because they fix the feature space before learning and can limit task-specific adaptation. Instead, a learned deep representation was prioritized.

Based on these considerations, ResNet18 was chosen as the backbone. It provides a good balance between representational power and computational cost. It is clearly stronger than the shallow CNN baselines reviewed earlier, while still being light enough for the available dataset size and hardware. The main idea behind the architecture is to combine information from two scales. In Arabic calligraphy, style is not determined by only one kind of visual cue. Some characteristics are global, such as spacing, rhythm, slant, overall proportions, and the general organization of the writing. Other characteristics are local, such as stroke curvature, thickness transitions, junction shapes, and edge endings. If the model uses only the deepest backbone feature, it may capture the general style, but lose some of the finer spatial details needed to separate visually similar classes. For this reason, the proposed model combines mid-level and deep-level features within the same backbone. The input image is first passed through the ResNet18 network. From this backbone, two feature maps are extracted: the layer3 output, which is used as the local branch, and the layer4 output, which is used as the global branch. The layer3 feature map is used for the local branch because it still preserves useful spatial detail, while already being more informative than low-level edge features. This makes it a good compromise for capturing fine stroke-level properties. In this branch, the feature map is first passed through a 1×1 convolution, followed by Batch Normalization and a ReLU activation. Global Average Pooling is then applied to obtain a compact local descriptor. The layer4 feature map is used for the global branch. Since this is the deepest feature map in the backbone, it carries more semantic and structural information about the writing as a whole. Global Average Pooling is applied directly to this feature map to produce a global descriptor. The local and global descriptors are then concatenated into a single fused representation. This fused vector is passed to a lightweight classification head made of a fully connected layer with 256 hidden units, a ReLU activation, a dropout layer with probability 0.3, and a final fully connected layer that maps to the style classes. The model outputs class logits. During training, these logits are optimized using cross-entropy loss. During inference, they can be converted into probabilities using softmax, and the predicted class is taken as the one with the highest score. Noting that the model does not rely on

two separate CNN feature extractors. Instead, both the local and global branches come from different stages of the same ResNet18. This keeps the model efficient while still allowing it to capture complementary information at different levels. Overall, this architecture was chosen because it addresses three practical needs at the same time: it is stronger than the shallow baseline models, it captures both local stroke-level details and broader global style structure, and it remains feasible to train with the available data and hardware. Fig6 represents the model architecture as described previously

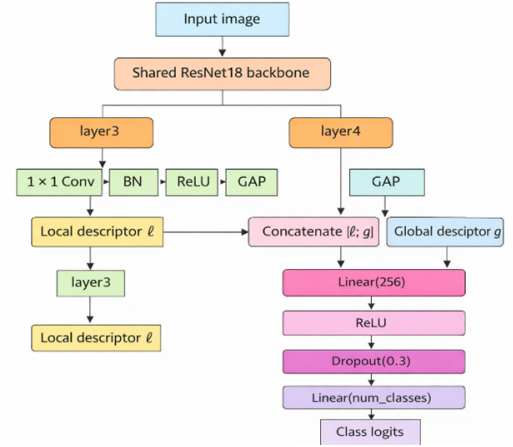


Fig. 6: Model Architecture

V. TRAINING AND EVALUATION STRATEGY

The proposed model was trained using a two-stage transfer learning strategy. In the warmup stage, the pretrained ResNet18 backbone was frozen, and only the newly added local-global fusion layers were trained for 8 epochs. In the fine-tuning stage, the higher backbone layers (layer3 and layer4) were unfrozen and trained for up to 6 additional epochs, while the lower layers remained fixed. This strategy was adopted to adapt the pretrained representation gradually while reducing the risk of overfitting. Training used the AdamW optimizer with differential learning rates. In the initial stage, the backbone learning rate was set to $1e-4$, while the newly added head used $5e-4$. During fine-tuning, these were reduced to $2e-5$ for the backbone and $2e-4$ for the head. A cosine annealing learning-rate schedule was applied, with weight decay set to $1e-4$. The classifier head used dropout with probability 0.3, and the loss function was cross-entropy with label smoothing 0.05. To reduce the effect of class imbalance, class-weighted training was also used. The model was trained with input images resized to 224×224 and a batch size of 8. Early stopping was enabled using validation loss, with patience 2 and a minimum improvement threshold of 0.0015. During training, checkpoints were saved for the latest model, the best model according to validation macro F1, and the best model according to validation loss. Model performance was evaluated after each epoch using accuracy, macro precision, macro recall, and macro F1-score. Macro F1 was treated as the

main performance indicator because it gives equal importance to all classes, making it more suitable than accuracy alone under class imbalance. In addition, confusion matrices and per-class metrics were used to analyze which styles were most often confused. All experiments were conducted on a personal laptop equipped with an NVIDIA GeForce RTX 3050 Laptop GPU and 16 GB RAM. This hardware setup influenced the training configuration, particularly the use of batch size 8 and a relatively lightweight ResNet18 backbone.

VI. EXPERIMENTS AND RESULTS

Two experiments were retained for comparison. Both used the same ResNet18 local-global fusion architecture, while differing in the fine-tuning strategy. The first experiment was the initial grouped-split run. 7 shows the grouped-split loss curves, it achieved the best raw result, with 96.55% accuracy and 95.23% F1-score (8). However, its training curves showed that after fine-tuning began, training loss kept decreasing while validation loss stopped improving and later increased, indicating late-stage overfitting.

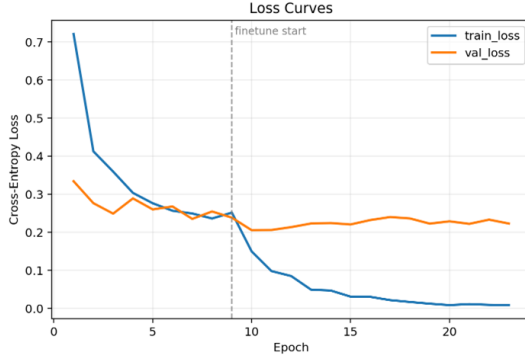


Fig. 7: Grouped-split run train and evaluation loss curves

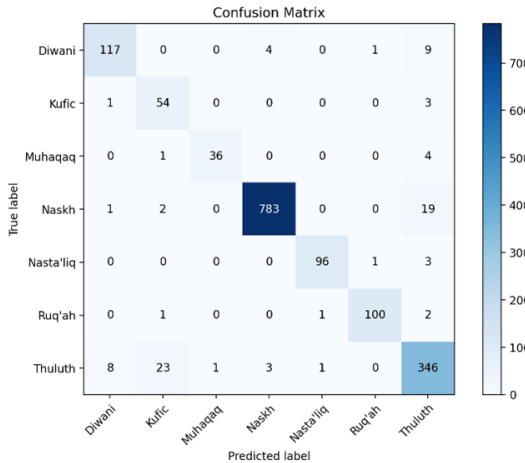


Fig. 8: Grouped-split run confusion matrix

The second experiment used a more stable fine-tuning strategy. 9 shows the stable fine-tuning run loss curves, The architecture was unchanged, but fine-tuning was made more

conservative through partial backbone unfreezing, lower learning rates, BatchNorm freezing, and earlier stopping. This run achieved 96.11% accuracy and 94.86% F1-score (10). Although slightly lower in score, it showed better training stability.

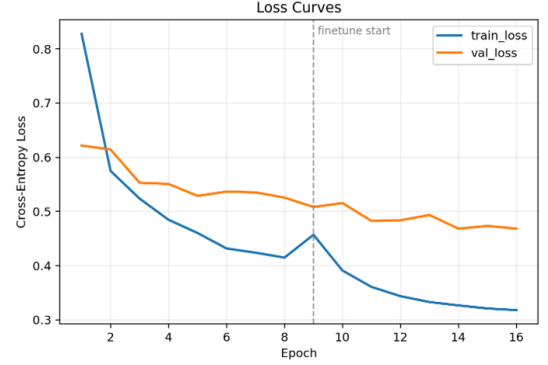


Fig. 9: Stable fine tuning run train and evaluation loss curves

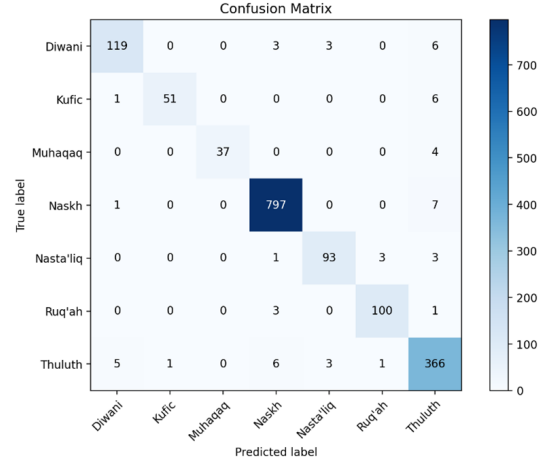


Fig. 10: Stable fine tuning run confusion matrix

These experiments show that the architecture itself was effective in both cases. The main difference was not the model design, but the aggressiveness of the fine-tuning stage. The loss curves of both runs are included to compare optimization behavior. The confusion matrices are also shown for both experiments in order to compare class-level prediction patterns. grouped-split confusion matrix stable fine-tuning (fix1) confusion matrix In both cases, the strongest confusions appeared among visually similar classes such as Diwani, Kufic, and Thuluth.

VII. CHALLENGES AND FIXE

- **Class imbalance across all source datasets:** Naskh dominates HICMA, and similar skews exist in Duwat-Bench and Ruf'a. Addressed through the class-aware augmentation strategy (minority over-sampling, majority light augmentation) and class-weighted cross-entropy loss

during training. Macro-F1 is used as the primary evaluation metric rather than accuracy to avoid inflated scores driven by dominant classes.

- **Mixed-style and unannotated samples:** Several images in the source datasets carried comma-separated (multi-style) labels or were present in a folder but absent from the associated CSV. These were silently discarded during the label-resolution step to retain only unambiguous single-style annotations.
- **Late-stage overfitting (Experiment 1):** Simultaneously unfreezing both `layer3` and `layer4` at standard learning rates caused training loss to keep decreasing while validation loss plateaued and then rose. Fixed in Experiment 2 through four targeted modifications: staged backbone unfreezing (`layer4` only first), reduced fine-tuning learning rates (2×10^{-5} backbone, 2×10^{-4} head), BatchNorm layer freezing during fine-tuning, and earlier stopping.
- **Persistent Diwani/Kufic/Thuluth confusion:** These three styles remain the primary source of misclassification across both experiments, due to genuine global visual similarity that requires discriminating on subtle stroke-level cues. Planned mitigations include stronger augmentation (elastic deformation already integrated) and attention-based modules to direct the model toward the most discriminative local regions.

VIII. CURRENT STATUS AND NEXT STEPS

A. Current Status

The style classification stage is complete and validated. The stable fine-tuning run achieves 96.11% accuracy and 94.86% macro F1-score on the seven-class merged dataset, with well-behaved training dynamics. The architecture, training protocol, and evaluation methodology are finalised. The primary remaining challenge on the classification side is reducing confusion among Diwani, Kufic, and Thuluth.

B. Next Steps: OCR Integration

The longer-term objective is a two-stage pipeline in which the style classifier feeds a downstream OCR module. As established in the literature review, no existing Arabic OCR system has been designed or evaluated for artistic calligraphy. These systems therefore serve only as baseline starting points that will require calligraphy-specific fine-tuning. A prerequisite for this is the paired calligraphic transcription dataset currently being assembled from the sources collected in this project. Once available, the following steps will be pursued.

1) *Survey of Candidate Arabic OCR Models:* The following models were reviewed and are considered candidate backbones for the OCR stage:

a) *Tesseract OCR [9]:* The most widely used open-source OCR engine. Since version 4 it uses an LSTM-based recognition pipeline and ships a dedicated Arabic model (`ara.traineddata`). CPU-friendly and appropriate for clean printed Arabic text. However, it was not designed for

handwritten or calligraphic input and is expected to degrade significantly on artistic calligraphy without retraining.

b) *EasyOCR [10]:* A Python-based deep-learning library supporting 80+ languages including Arabic. Uses a CRAFT text detector and a CRNN (CNN + BiLSTM + CTC) recogniser. More robust than Tesseract on diverse fonts, but similarly untested on artistic calligraphy. PaddleOCR is a competitive alternative in the same category and may be evaluated alongside EasyOCR.

c) *TrOCR [11]:* A transformer encoder-decoder model (BEiT encoder + RoBERTa decoder) originally trained on English printed and handwritten text. Represents the state of the art for general OCR and is the most promising starting point for fine-tuning, due to its strong attention-based sequence modelling capabilities.

d) *HATFormer [12]:* The closest existing work to our target domain: a TrOCR-based model fine-tuned for historical handwritten Arabic manuscripts, with Arabic-specific tokenisation and post-processing normalisation. Even so, historical manuscripts differ from artistic calligraphy in stroke deformation and stylistic variety, so further domain adaptation will be required.

e) *Qalam [14]:* A recent multimodal model (Swinv2 encoder + RoBERTa decoder) designed specifically for Arabic OCR and handwriting recognition. Represents the current state of the art for Arabic-specific recognition but requires substantially more compute than TrOCR-based alternatives.

2) *Proposed Classifier → OCR Pipeline:* As shown in Fig11, the proposed two-stage pipeline operates as follows:

- 1) **Stage 1 — Style classifier:** The ResNet18 local-global fusion model classifies the input image into one of the seven styles. The predicted style label is passed to Stage 2.
- 2) **Stage 2 — Style-conditioned OCR:** An OCR model transcribes the text. The style label from Stage 1 is used either to (a) select a style-specific fine-tuned OCR checkpoint, or (b) condition a shared model via a style embedding appended to the encoder input.

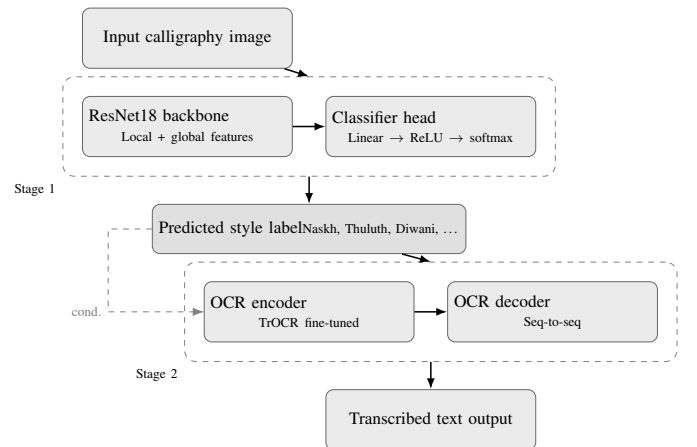


Fig. 11: Proposed two-stage pipeline.

The initial implementation will target TrOCR fine-tuned on the paired calligraphic transcription dataset being constructed. Performance will be measured using Character Error Rate (CER) and Word Error Rate (WER).

3) *Calligraphy-Specific OCR Challenges*: Beyond the general Arabic OCR difficulties documented in the literature review [8], calligraphic manuscripts introduce additional obstacles:

- **Style-dependent stroke geometry**: Each calligraphy style deforms standard character shapes in stylistically consistent but OCR-unconventional ways. A model trained on one style will generalise poorly to another without style-specific adaptation — which is the core motivation for Stage 1.
- **Overlapping and broken strokes**: Calligraphic writing frequently produces horizontally overlapping strokes and decorative extensions that violate the segmentation assumptions of standard OCR pipelines.
- **Background texture and manuscript noise**: The datasets used preserve realistic manuscript backgrounds with age-related degradation, ink bleeding, and variable illumination, all of which reduce OCR reliability compared to clean digital typography.
- **Absence of tashkeel**: Diacritical marks are largely absent in manuscript calligraphy, increasing phonetic ambiguity and requiring stronger language-model post-processing to recover the intended text.
- **Limited paired training data**: No large publicly available dataset pairs calligraphic images with ground-truth transcriptions across multiple styles. This gap is being actively addressed: a paired dataset is currently being constructed from the calligraphic sources collected in this project, and completing it is the immediate next step before OCR fine-tuning can begin.

REFERENCES

- [1] UNESCO, “Unesco adds arabic calligraphy to intangible cultural heritage of humanity list,” <https://www.unesco.org/en/articles/unesco-adds-arabic-calligraphy-intangible-cultural-heritage-humanity-list>, accessed: 2026-02-15.
- [2] A. Ismail, Z. Kamel, and R. Mahmoud, “Hicma: The handwriting identification for calligraphy and manuscripts in arabic dataset,” in *Proceedings of ArabicNLP 2023*, 2023, pp. 24–32.
- [3] S. R. Allaf and R. Al-Hmouz, “Automatic recognition of artistic arabic calligraphy types,” *Journal of King Abdulaziz University*, vol. 27, no. 1, pp. 3–17, 2016.
- [4] A. K. Hussein, “Fast learning neural network based on texture for arabic calligraphy identification,” *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 21, no. 3, pp. 1794–1799, 2021.
- [5] Z. Kaoudja, M. L. Kherfi, and B. Khaldi, “A new computational method for arabic calligraphy style representation and classification,” *Applied Sciences*, vol. 11, no. 11, p. 4852, 2021.
- [6] R. Al-Hmouz, “Deep learning autoencoder approach: Automatic recognition of artistic arabic calligraphy types,” *Kuwait Journal of Science*, vol. 47, no. 3, 2020.
- [7] Z. Alyafeai, M. S. Al-shaibani, M. Ghaleb, and Y. A. Al-Wajih, “Calliar: An online handwritten dataset for arabic calligraphy,” *Neural Computing and Applications*, vol. 34, no. 23, pp. 20 701–20 713, 2022.
- [8] M. S. Kasem *et al.*, “Advancements and challenges in arabic optical character recognition: A comprehensive survey,” *ACM Computing Surveys*, 2024.
- [9] R. Smith, “An overview of the tesseract ocr engine,” in *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, vol. 2, 2007, pp. 629–633.
- [10] Jaidev AI, “EasyOCR,” <https://github.com/JaidevAI/EasyOCR>, 2020.
- [11] M. Li *et al.*, “TrOCR: Transformer-based optical character recognition with pre-trained models,” *arXiv preprint arXiv:2109.10282*, 2023.
- [12] A. Chan, A. Mijar, M. Saeed, C.-W. Wong, and A. Khater, “HATFormer: Historic handwritten arabic text recognition with transformers,” *arXiv preprint arXiv:2410.02179*, 2024.
- [13] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, “Albumentations: Fast and flexible image augmentations,” *Information*, vol. 11, no. 2, p. 125, 2020.
- [14] G. Bhatia, E. M. B. Nagoudi, F. Alwajih, and M. Abdul-Mageed, “Qalam: A multimodal LLM for arabic optical character and handwriting recognition,” in *Proceedings of ArabicNLP 2024*, 2024.